

**Lincoln University College**

**Faculty of Computer Science and Multimedia**

**PocketDRS: A Single-View 3D Trajectory  
Reconstruction and Decision Review System for  
Cricket**

**A PROJECT REPORT**

Submitted to

Department of Information Technology

Phoenix College of Management

Maitidevi, Kathmandu

In partial fulfilment of the requirements for the Bachelor in Information  
Technology (BIT)

Submitted by

**Niraj Kafle**

BIT 7th Semester

University ID : LC0003001674

**2026/05/30**

# DECLARATION

I hereby declare that this report is an original work prepared by me for the requirement of the Bachelor in Information Technology programme under the supervision of Mr. Saishab Bhattarai. All sources and references used have been duly acknowledged. The report has not been previously submitted to any institution for any degree or diploma.

Date: \_\_\_\_\_

Signature: \_\_\_\_\_

Name: Niraj Kifle



**Lincoln University College**  
**Faculty of Computer Science and Multimedia**  
**Phoenix College of Management**  
**Maitidevi, Kathmandu**  
**Bachelors in Information Technology (BIT)**

**LETTER OF APPROVAL**

The project report entitled “**PocketDRS: A Single-View 3D Trajectory Reconstruction and Decision Review System for Cricket**” submitted by **Niraj Kafle** (University ID: **LC0003001674**), in partial fulfilment of the requirements for the Bachelor in Information Technology (BIT), has been accepted as a record of work independently carried out under supervision.

In our opinion, it is satisfactory in scope and quality as a project for the required degree.

**Er. Santosh Dhungana**  
*Head of Department*

**Mr. Bijay Mishra**  
*Program Coordinator*

**Mr. Saishab Bhattarai**  
*Internal Examiner*

**External Examiner**

# ABSTRACT

PocketDRS is a low-cost Leg Before Wicket (LBW) decision-support system for cricket that works with a single smartphone camera. Professional systems such as Hawk-Eye need six to eight synchronised high-speed cameras and dedicated hardware, which puts them out of reach of schools, clubs, and training grounds. PocketDRS shows that a careful single-view workflow can still produce practical decision support at those venues. A Flutter app records the delivery and lets the user mark the pitch corners and both sets of stumps. A FastAPI server then detects the ball from motion, colour and a learned detector, links the detections into one smooth path using Random Sample Consensus (RANSAC), recovers the camera pose from the marked stumps, and rebuilds the ball trajectory in real-world three-dimensional (3D) space using projectile-motion physics. A rule-based engine applies the three LBW checks of the International Cricket Council (ICC) Rule 36. On a controlled synthetic harness of eight ground-truth deliveries the system recovered the camera position correctly in every scene and returned the correct verdict in all eight cases (8/8); on a real handheld off-spin clip it produced a plausible end-to-end NOT OUT verdict within honest monocular bounds, and on an unsuitable real clip it safely declined to invent a decision.

**Keywords:** Cricket, Decision Review System, Monocular 3D Reconstruction, Computer Vision, LBW.

## ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who contributed to the completion of this project. Their guidance and continuous support have been invaluable throughout this work.

I am profoundly grateful to the Principal of Phoenix College of Management, Prof. Dr. Fatta Bahadur KC, for his inspiring leadership and academic guidance.

My deepest appreciation goes to my project supervisor, Mr. Saishab Bhattarai, for his invaluable guidance, constructive feedback, and continuous support. His expertise and insightful suggestions greatly contributed to the success of this work.

I am also thankful to the Department of Information Technology at Phoenix College of Management for the academic environment and institutional support, and to the faculty members for their suggestions during the course of this work.

Finally, I express my deepest gratitude to my family and friends for their unwavering support, motivation, and encouragement throughout this journey.

# CERTIFICATE

This is to certify that the student Niraj Kafle has completed the Bachelor in Information Technology Final Year Project titled “PocketDRS: A Single-View 3D Trajectory Reconstruction and Decision Review System for Cricket” under the supervision of Mr. Saishab Bhattarai at Lincoln University College, Faculty of Computer Science and Multimedia. The student has successfully implemented a working prototype, conducted testing, and documented the system according to academic standards. This certificate is issued on completion of all project requirements.

Issued at: Phoenix College of Management, Maitidevi, Kathmandu

Date: May, 2026

# TABLE OF CONTENTS

|                                                 |             |
|-------------------------------------------------|-------------|
| <b>Declaration</b>                              | <b>i</b>    |
| <b>Letter of Approval</b>                       | <b>ii</b>   |
| <b>Abstract</b>                                 | <b>iii</b>  |
| <b>Acknowledgement</b>                          | <b>iv</b>   |
| <b>Certificate</b>                              | <b>v</b>    |
| <b>List of Figures</b>                          | <b>viii</b> |
| <b>List of Tables</b>                           | <b>ix</b>   |
| <b>List of Abbreviations</b>                    | <b>xi</b>   |
| <b>1 Introduction</b>                           | <b>1</b>    |
| 1.1 Introduction . . . . .                      | 1           |
| 1.2 Problem Statement . . . . .                 | 1           |
| 1.3 Objectives . . . . .                        | 2           |
| 1.3.1 General Objective . . . . .               | 2           |
| 1.3.2 Specific Objectives . . . . .             | 2           |
| 1.4 Scope and Limitation . . . . .              | 2           |
| 1.4.1 Scope . . . . .                           | 2           |
| 1.4.2 Limitation . . . . .                      | 3           |
| 1.5 Development Methodology . . . . .           | 3           |
| 1.6 Report Organisation . . . . .               | 3           |
| <b>2 Background Study and Literature Review</b> | <b>5</b>    |
| 2.1 Background Study . . . . .                  | 5           |
| 2.2 Literature Review . . . . .                 | 6           |
| <b>3 System Analysis and Design</b>             | <b>8</b>    |
| 3.1 System Analysis . . . . .                   | 8           |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 3.1.1    | Requirement Analysis . . . . .                                           | 8         |
| 3.1.2    | Feasibility Analysis . . . . .                                           | 9         |
| 3.1.3    | Data Modelling: ER Diagram . . . . .                                     | 10        |
| 3.1.4    | Process Modelling: Data Flow Diagram (DFD) . . . . .                     | 11        |
| 3.2      | System Design . . . . .                                                  | 12        |
| 3.2.1    | Architectural Design . . . . .                                           | 12        |
| 3.2.2    | Database Schema Design . . . . .                                         | 13        |
| 3.2.3    | Interface Design (UI/UX) . . . . .                                       | 14        |
| 3.2.4    | Physical DFD . . . . .                                                   | 15        |
| 3.3      | Algorithm Details . . . . .                                              | 15        |
| 3.3.1    | Step 1: Finding Where the Camera Was Standing . . . . .                  | 16        |
| 3.3.2    | Step 2: Finding the Ball in Each Frame . . . . .                         | 16        |
| 3.3.3    | Step 3: Joining the Detections into One Ball Path . . . . .              | 17        |
| 3.3.4    | Step 4: Rebuilding the Ball Path in Real-World 3D . . . . .              | 17        |
| 3.3.5    | Step 5: Making the LBW Decision (ICC Rule 36) . . . . .                  | 17        |
| <b>4</b> | <b>Implementation and Testing</b>                                        | <b>19</b> |
| 4.1      | Implementation . . . . .                                                 | 19        |
| 4.1.1    | Tools Used . . . . .                                                     | 19        |
| 4.1.2    | Implementation Details of Modules . . . . .                              | 19        |
| 4.1.3    | Robustness and Edge-Case Hardening . . . . .                             | 20        |
| 4.2      | Testing . . . . .                                                        | 21        |
| 4.2.1    | Test Cases for Unit Testing . . . . .                                    | 21        |
| 4.2.2    | Test Cases for System Testing . . . . .                                  | 23        |
| 4.2.3    | Synthetic Validation . . . . .                                           | 24        |
| 4.2.4    | Real-World Validation . . . . .                                          | 27        |
| 4.2.5    | Real-World Validation: A Successfully Tracked Delivery (test3) . . . . . | 29        |
| <b>5</b> | <b>Conclusion and Future Recommendations</b>                             | <b>33</b> |
| 5.1      | Conclusion . . . . .                                                     | 33        |
| 5.2      | Lesson Learnt/Outcome . . . . .                                          | 33        |
| 5.3      | Future Recommendations . . . . .                                         | 34        |
|          | <b>References</b>                                                        | <b>34</b> |

|                  |    |
|------------------|----|
| Appendices       | 37 |
| Project Log Book | 39 |

## LIST OF FIGURES

|     |                                                                                                                                                                                                                                                                                     |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | PocketDRS entity–relationship diagram. . . . .                                                                                                                                                                                                                                      | 11 |
| 3.2 | Level 0 (context) DFD. . . . .                                                                                                                                                                                                                                                      | 11 |
| 3.3 | Level 1 DFD. . . . .                                                                                                                                                                                                                                                                | 12 |
| 3.4 | Three-tier system architecture of PocketDRS. . . . .                                                                                                                                                                                                                                | 13 |
| 3.5 | Result screen for OUT (left), NOT OUT (middle), UMPIRE’S CALL<br>(right). . . . .                                                                                                                                                                                                   | 15 |
| 3.6 | Physical DFD of the deployed system. . . . .                                                                                                                                                                                                                                        | 15 |
| 4.1 | A sample frame from the synthetic harness: full ICC pitch, crease lines,<br>and the simulated red ball used to generate ground-truth deliveries. . .                                                                                                                                | 25 |
| 4.2 | Synthetic sweep summary: per-scene speed, bounce, and impact errors<br>plotted against the monocular pass thresholds. . . . .                                                                                                                                                       | 27 |
| 4.3 | Frames from the real-world test clip. . . . .                                                                                                                                                                                                                                       | 28 |
| 4.4 | Broadcast-style overlay on <code>test3.mp4</code> : the recovered ball path, the<br>bounce marker, the impact point on the pads, and the predicted contin-<br>uation to the stumps. . . . .                                                                                         | 31 |
| 4.5 | Left: full tracked ball path on <code>test3.mp4</code> . Right: the same delivery<br>in the in-app interactive 3D Hawk-Eye viewer (WebGL), showing the<br>reconstructed flight and predicted path, the pitch and stumps, the LBW<br>corridor, and the live decision panels. . . . . | 32 |

## LIST OF TABLES

|     |                                                                                                                                                                   |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Functional requirements of the PocketDRS system. . . . .                                                                                                          | 8  |
| 3.2 | Non-functional requirements of the PocketDRS system. . . . .                                                                                                      | 9  |
| 3.3 | Firestore collections used by PocketDRS. . . . .                                                                                                                  | 14 |
| 4.1 | Unit test cases covering the calibration, tracking, trajectory, reconstruction, LBW, hardening, and API modules. . . . .                                          | 21 |
| 4.2 | System (end-to-end) test cases. . . . .                                                                                                                           | 23 |
| 4.3 | Per-scene results for the eight synthetic scenarios. All eight pass the monocular bounds (speed $\pm 55$ km/h, bounce $\pm 175$ cm, impact $\pm 110$ cm). . . . . | 27 |

## LIST OF ABBREVIATIONS

|        |                                    |
|--------|------------------------------------|
| API    | Application Programming Interface  |
| BIT    | Bachelor in Information Technology |
| CV     | Computer Vision                    |
| DFD    | Data Flow Diagram                  |
| DRS    | Decision Review System             |
| EKF    | Extended Kalman Filter             |
| ER     | Entity-Relationship                |
| FOV    | Field of View                      |
| FPS    | Frames Per Second                  |
| HSV    | Hue, Saturation, Value             |
| HTTP   | Hypertext Transfer Protocol        |
| ICC    | International Cricket Council      |
| JSON   | JavaScript Object Notation         |
| LBW    | Leg Before Wicket                  |
| MOG2   | Mixture of Gaussians 2             |
| PnP    | Perspective-n-Point                |
| POV    | Point of View                      |
| RANSAC | Random Sample Consensus            |
| REST   | Representational State Transfer    |
| RMS    | Root Mean Square                   |
| ROI    | Region of Interest                 |
| SDK    | Software Development Kit           |
| UI/UX  | User Interface / User Experience   |
| UID    | User Identifier                    |
| UML    | Unified Modeling Language          |
| URI    | Uniform Resource Identifier        |
| YOLO   | You Only Look Once                 |

# Chapter 1:

## INTRODUCTION

### 1.1 Introduction

Modern cricket relies heavily on technology to review close decisions, and Leg Before Wicket (LBW) is one of the hardest of those decisions for the human eye. At the top level the game uses Hawk-Eye, which reconstructs the ball trajectory from six to eight synchronised high-speed cameras and dedicated processing hardware. Hawk-Eye is accurate, but the equipment, installation, and operating cost place it well beyond the reach of schools, clubs, local leagues, and coaching grounds, where the same close decisions occur every week with no review support at all.

PocketDRS addresses this gap. It is a decision-support system that runs on a normal smartphone and a lightweight cloud service. The user records or picks a short clip of the delivery, taps the four corners of the visible pitch to calibrate, sends the clip to the server, and gets back a 3D visualisation together with an LBW verdict. The aim is practical decision support at low cost, not broadcast-grade precision; the system is therefore positioned as an aid to umpiring rather than a replacement for it.

The contribution of the project is to show that a carefully constrained single-camera pipeline, built around the known geometry of a cricket pitch and the physics of projectile motion, can produce consistent and human-readable LBW review support end-to-end. The working prototype demonstrates this with a complete mobile-to-cloud system that combines computer vision, geometric reconstruction, and cricket-specific decision logic.

### 1.2 Problem Statement

Professional Decision Review System (DRS) installations depend on multiple synchronised cameras to recover depth and reconstruct the ball trajectory in three dimensions. This produces strong results, but the required equipment, installation effort, and operating cost make such systems inaccessible outside major venues.

When the same decision-review problem is shifted to a single consumer camera, several technical and practical difficulties arise:

1. A single camera cannot directly see depth, so the system has to recover distance indirectly, for example from how big the ball looks in the image or from how its motion follows the laws of physics.

2. A cricket ball is small, fast, and motion-blurred, and can easily be confused with background objects, players, shadows, or spectators.
3. Small errors in marking the pitch corners produce much larger errors at the far end of the pitch, because the far end occupies only a few pixels.
4. Lighting changes, camera shake, the bowler or batter blocking the view, and inconsistent recording conditions all make a real ground less predictable than a controlled broadcast environment.

Therefore, the central problem addressed by this project is whether a single-phone-camera workflow, supported by pitch calibration and lightweight computer-vision techniques, can produce a sufficiently consistent and understandable LBW review aid for ordinary cricket grounds.

## **1.3 Objectives**

### **1.3.1 General Objective**

To design and implement a low-cost cricket decision-support system that uses a single camera view to analyse a delivery, estimate the ball path in calibrated pitch coordinates, and assist LBW review.

### **1.3.2 Specific Objectives**

1. To develop a user-guided calibration process that marks the pitch corners and stumps to recover the camera pose.
2. To detect and track the ball from a single-camera delivery video and reconstruct its path in real-world 3D using physics-based fitting.
3. To estimate an LBW outcome using rule-based checks on the pitching line, impact line, and projected stump contact.
4. To present the result through an interactive 3D visualisation that helps users interpret the decision.

## **1.4 Scope and Limitation**

### **1.4.1 Scope**

The PocketDRS project encompasses the complete workflow from video capture to LBW decision:

- Mobile application for video selection, trimming, and pitch calibration.
- Backend processing pipeline for ball detection, tracking, and 3D reconstruction.
- LBW decision engine implementing ICC Rule 36.
- Interactive 3D visualisation of the trajectory.
- Local and cloud-based persistent storage of pitches and analyses.

#### 1.4.2 Limitation

- The system operates under umpire-POV camera placement; other angles may require retraining or recalibration.
- Accuracy degrades with heavy ball occlusion, poor lighting, or unstable camera mounting.
- The system is not ICC-certified and is designed for decision support rather than final authority.
- Monocular depth estimation remains fundamentally limited compared to multi-camera triangulation.
- Performance depends on proper pitch calibration; user error in marking reference points reduces decision reliability.

### 1.5 Development Methodology

PocketDRS followed an incremental, agile approach: requirement analysis (from literature and existing systems), proof-of-concept on calibration / detection / 3D reconstruction, iterative development of the full pipeline with continuous testing, unit and synthetic-harness validation, and deployment of the backend on Google Cloud Run with Firestore persistence alongside the mobile app on Android and iOS.

### 1.6 Report Organisation

The report is organised into five chapters:

- **Chapter 1: Introduction** introduces the project, the problem statement, the objectives, the scope and limitations, the development methodology, and the organisation of the report.

- **Chapter 2: Background Study and Literature Review** presents the underlying theory (camera geometry, projectile motion, RANSAC, ICC Rule 36) and reviews the related work.
- **Chapter 3: System Analysis and Design** covers the functional and non-functional requirements, feasibility analysis, ER model, data flow diagrams, system architecture, and the five-step algorithm.
- **Chapter 4: Implementation and Testing** details the implementation of the modules, the unit and system tests, the synthetic validation, and the real-world clips.
- **Chapter 5: Conclusion and Future Recommendations** summarises the outcome, the lessons learnt, and the recommendations for future work.

The references and the appendices follow Chapter 5.

## Chapter 2:

# BACKGROUND STUDY AND LITERATURE REVIEW

### 2.1 Background Study

**Pinhole Camera Model and Intrinsics.** A pinhole camera is the simplest model of any camera: light goes through one tiny hole and lands on a flat sensor behind it. A smartphone lens behaves close enough to this model for our purposes. The four numbers that describe the camera itself (focal length  $f_x, f_y$  and principal point  $c_x, c_y$ ) are packed into a  $3 \times 3$  matrix called  $K$ , which converts a 3D world point into a pixel. Zhang’s checkerboard method [2] is the standard way to measure  $K$ . PocketDRS skips the checkerboard and estimates  $K$  from the frame size and a typical phone horizontal field of view of  $67^\circ$ .

**Camera Pose (PnP).** The camera pose is simply where the camera is standing and which way it is pointing, written as a rotation  $R$  and a translation  $t$ . The Perspective-n-Point algorithm [7] works out  $R$  and  $t$  from a handful of known 3D points and the pixels they appear at. It finds the pose that makes those known points line up best with the image. PocketDRS feeds it the four tapped pitch corners.

**Homography.** A homography is a  $3 \times 3$  matrix  $H$  that links two different views of the same flat surface [5]. Because a cricket pitch is flat, one homography fits the whole pitch. Once  $H$  is known from the four corner taps, any pitch pixel can be converted to a real position in metres. RANSAC [3] throws away wrong point matches before  $H$  is fitted.

**Projectile Motion.** A thrown ball flies in a predictable curve, set by gravity, because the wind drag in a short delivery is small enough to ignore. Its position obeys

$$X(t) = X_0 + V_x t, \quad Y(t) = Y_0 + V_y t, \quad Z(t) = Z_0 + V_z t - \frac{1}{2} g t^2,$$

with  $g = 9.81 \text{ m/s}^2$ . The ball moves at a steady speed sideways and forward, and falls under gravity. Any candidate path that does not follow this shape cannot be a real cricket ball. The bounce is treated as the vertical speed flipping sign and shrinking by a fixed factor (coefficient of restitution  $\approx 0.55$ ).

**RANSAC.** RANSAC is a way to fit a model when most of the data is wrong [3]. It picks a tiny random sample, fits a candidate model to it, and counts how many other

points agree. After many tries it keeps the model with the most agreement. PocketDRS uses RANSAC twice: once to fit the homography and once to link the per-frame ball detections into one path.

**Kalman Filter (for reference).** A Kalman filter is the textbook way to track a moving object frame by frame. PocketDRS does not use one. RANSAC plus a single projectile-curve fit handled clutter better and was easier to explain. An Extended Kalman Filter remains the obvious upgrade for live tracking.

**ICC Rule 36 (LBW).** Law 36 of cricket [17] sets the LBW rule. Three things must all be true at once: the ball pitches in line with the stumps, strikes the pad in line, and would have gone on to hit the stumps. A small  $\approx 25$  mm margin around the stump edge is left to the on-field umpire (“umpire’s call”). PocketDRS checks all three conditions and respects the margin.

## 2.2 Literature Review

Owens *et al.* [1] described the original Hawk-Eye ball tracker. It uses six to eight synchronised cameras and custom calibration to rebuild ball paths to a few millimetres. It is the benchmark PocketDRS is compared against, and the source of the multi-camera cost problem the project tries to side-step.

Zhang [2] introduced a flexible camera-calibration method using a printed checkerboard. It removed the need for expensive calibration rigs and is still the textbook way to measure focal length and principal point. The same intrinsic-parameter model is what  $K$  in our pinhole equation refers to, even though we estimate  $K$  from FOV instead of running the full checkerboard.

Fischler and Bolles [3] proposed RANSAC, a way to fit a model when many of the data points are wrong. It samples small random subsets, fits a candidate, and keeps the one that explains the most points. PocketDRS uses RANSAC twice, once for the pitch homography and once to link noisy per-frame ball detections into one trajectory. Hartley and Zisserman [5] wrote the standard reference on multiple-view geometry, including homography and projective reconstruction. The single-view geometry used to map pitch pixels to metres comes straight from this text.

Triggs *et al.* [4] formalised bundle adjustment as the joint least-squares refinement of camera parameters and 3D structure. The projectile-fit step uses the same least-squares idea, only with a physics-based parametric curve instead of a free 3D point cloud.

Ponglertnapakorn and Suwajanakorn [6] recovered 3D ball trajectories from a single 2D track by using the physics of ball motion as the constraint. This is the closest published prior work and is the direct evidence that monocular 3D ball reconstruction

is feasible at all.

Bochinski *et al.* [8] showed that high-quality multi-frame tracking is possible by linking detections frame-to-frame using simple bounding-box overlap (IoU). The same detection-linking idea is used here, with a projectile-motion model in place of plain box overlap to reject background clutter.

**Synthesis.** Single-view sports analysis becomes possible when two things hold: the scene contains a known geometric reference (the pitch) and the motion follows a strong rule (projectile flight). PocketDRS uses both, and adds the mobile capture, cloud processing, and visualisation needed to make it usable end-to-end.

## Chapter 3:

# SYSTEM ANALYSIS AND DESIGN

### 3.1 System Analysis

#### 3.1.1 Requirement Analysis

##### Functional Requirements

**Table 3.1: Functional requirements of the PocketDRS system.**

| Requirement             | Description                                                                                                                          |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| User Authentication     | Users sign in via Firebase Authentication with email/-password or social login.                                                      |
| Video Capture/Selection | Users can record a new video or select an existing video from the gallery.                                                           |
| Video Trimming          | Users trim the selected video to isolate the delivery of interest.                                                                   |
| Pitch Calibration       | Users calibrate by marking the pitch corners and both sets of stumps (base and top); the camera pose is anchored on the stump marks. |
| Calibration Validation  | The system validates calibration quality and alerts users if quality is poor.                                                        |
| Save Pitch Config       | Calibration data is saved locally and to Firestore for reuse on the same ground.                                                     |
| Job Submission          | The user submits the trimmed video and calibration for server-side processing.                                                       |
| Ball Detection          | The server detects ball candidates using motion and colour cues.                                                                     |
| 3D Reconstruction       | The server recovers the ball trajectory in world coordinates using physics fitting.                                                  |
| LBW Decision            | The server evaluates and returns a decision (OUT / NOT OUT / UMPIRE'S CALL) with rationale.                                          |
| 3D Visualisation        | The client displays an interactive 3D trajectory using Three.js in a WebView.                                                        |
| Decision History        | Users view past analyses and decisions saved to Firestore.                                                                           |

---

The table above specifies the functional requirements of the system. As the project follows the structured approach, these requirements are modelled in detail by the data flow diagrams that follow, rather than by the object-oriented class, sequence, state, or deployment diagrams.

## Non-functional Requirements

**Table 3.2: Non-functional requirements of the PocketDRS system.**

| ID     | Requirement     | Description                                                                                                                                                                                                   |
|--------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-01 | Response Time   | Processing should complete within 30 seconds for a 5-second delivery clip.                                                                                                                                    |
| NFR-02 | Accuracy        | Reconstructed impact position within $\pm 110$ cm and bounce within $\pm 175$ cm of ground truth on the synthetic harness (honest monocular bounds); structural correctness on real handheld video confirmed. |
| NFR-03 | Offline Support | Pitch calibration and video selection work offline; analysis requires connectivity.                                                                                                                           |
| NFR-04 | Security        | User authentication and data encryption in transit; Firebase rules enforce privacy.                                                                                                                           |
| NFR-05 | Cross-Platform  | App runs on Android 6.0+ and iOS 12.0+ via Flutter.                                                                                                                                                           |
| NFR-06 | Usability       | First-time users can complete a full analysis within 5 minutes with minimal guidance.                                                                                                                         |
| NFR-07 | Maintainability | Code is modular and documented to support future enhancements.                                                                                                                                                |
| NFR-08 | Data Privacy    | User data is not shared with third parties; deletion requests are honoured.                                                                                                                                   |

### 3.1.2 Feasibility Analysis

#### Technical Feasibility

The system uses well-known, mature tools. Flutter handles the mobile app on Android

and iOS. FastAPI runs the backend. OpenCV, NumPy, and SciPy do the computer-vision work. Firebase handles sign-in and cloud storage. A working prototype already runs the full upload-to-result flow on ordinary phones, so the stack is proven.

### **Operational Feasibility**

The workflow is short and repeatable. The user places a phone behind the bowler-end stumps, records the delivery, taps four pitch corners to calibrate, trims the clip, and waits for the result. No training is needed beyond a short on-screen guide. It fits coaching nets, schools, and clubs.

### **Economic Feasibility**

The cost is essentially the phone and a tripod. There are no camera arrays, no licensed broadcast hardware, and no paid SDKs; the software is open source and the cloud runs on a free-tier-friendly Cloud Run service. A professional DRS install runs into hundreds of thousands of dollars; PocketDRS runs for the price of any modern smartphone.

### **Schedule Feasibility**

The project fits one semester. Requirements and design took weeks 1–3, the pipeline and app took weeks 4–12, and validation and the report took the final three weeks. Each module was built and tested on its own before being wired together, which kept slippage local.

#### **3.1.3 Data Modelling: ER Diagram**

The data model has six entities. A *User* (Firebase UID) owns many *Pitches* and submits many *Analyses*. Each *Pitch* has one *CalibrationProfile* (the saved corner and stump taps). Each *Analysis* uses one *Pitch*, holds one *DeliveryVideo* (the trimmed clip), and produces one *LBWResult*. Figure 3.1 shows the full diagram with attributes and crow’s-foot cardinalities.

In Firestore the same model lives under per-user collections: `users/{uid}` for the user document, `users/{uid}/pitches/{pitchId}` for saved calibrations, and `users/{uid}/analyses/{analysisId}` for completed analyses.

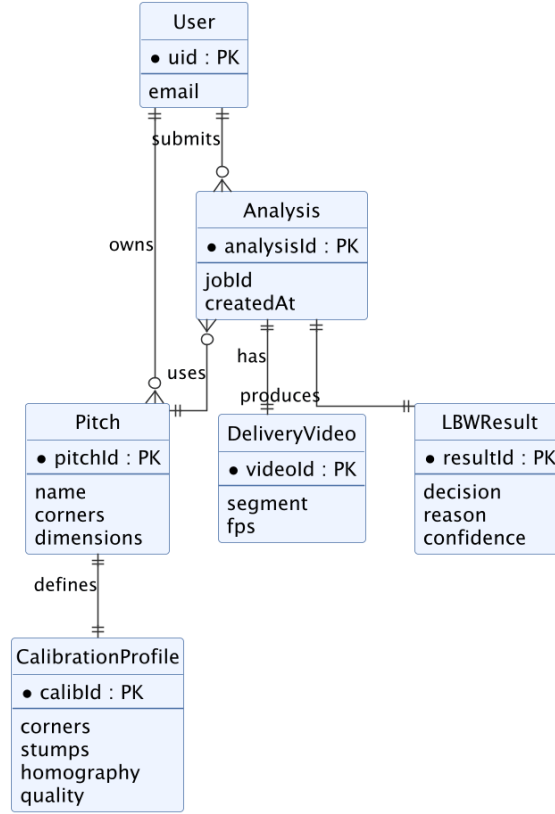


Figure 3.1: PocketDRS entity–relationship diagram.

### 3.1.4 Process Modelling: Data Flow Diagram (DFD)

The Level 0 DFD treats the whole system as one process. The User sends the clip and the calibration taps; Firebase exchanges sign-in tokens and stores the result; the system returns the LBW decision and the 3D view (Figure 3.2).



Figure 3.2: Level 0 (context) DFD.

The Level 1 DFD splits the system into four processes: (1) Capture & Trim Delivery on the mobile app, (2) Calibrate Pitch (collects corner and stump taps), (3) Analyse Delivery on the server (decode, detect ball, track, rebuild 3D, decide LBW), and (4) Present Result (3D view and verdict). Two data stores connect them: the Calibration Store (D1) and the Job/Result Store (D2). See Figure 3.3.

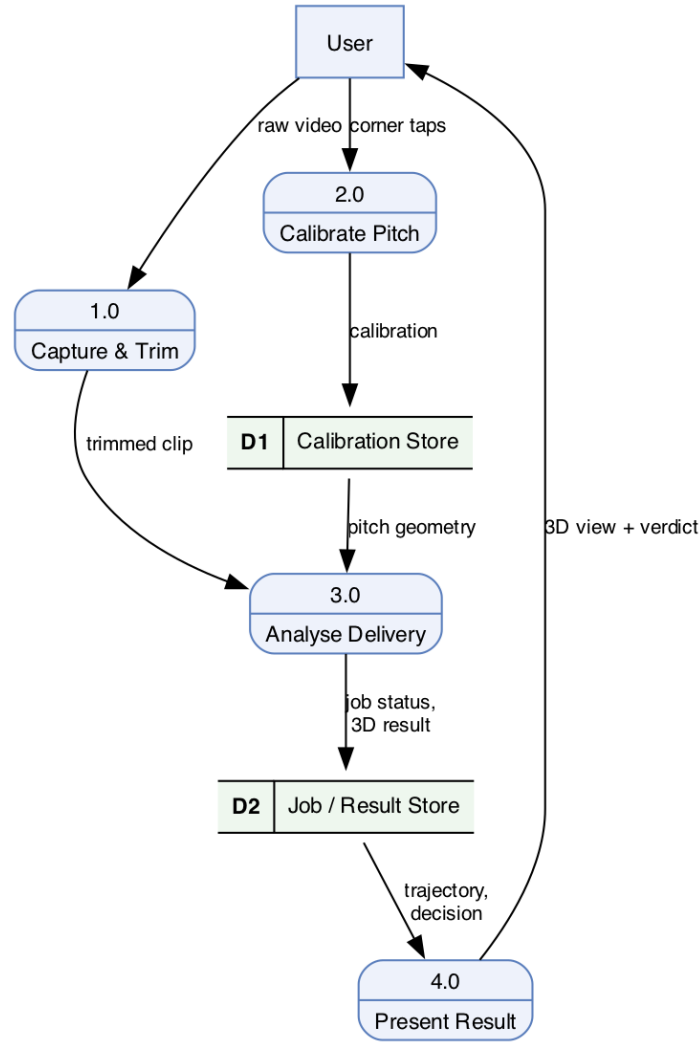


Figure 3.3: Level 1 DFD.

## 3.2 System Design

### 3.2.1 Architectural Design

PocketDRS follows a three-tier design: a client tier, a server tier (the API and the processing pipeline), and a storage tier. Figure 3.4 shows how they connect.

**Client.** The Flutter app on the phone handles sign-in, video recording or picking, trimming, pitch-corner tapping, job submission, and the embedded Three.js result viewer.

**API.** A FastAPI service exposes REST endpoints to upload a job, poll its status, and fetch the result. Uploads return immediately; the heavy work is queued and runs in the background so HTTP calls never block.

**Pipeline.** The pipeline is the computer-vision core (OpenCV, NumPy, SciPy, optional YOLO). It decodes the video, finds the ball, builds the trajectory, lifts it to 3D, and

applies the LBW rule. It writes progress to Firestore as it goes.

**Storage.** Firebase Authentication holds user identity. Firestore stores user, pitch, analysis, and live job documents. Cloud Storage holds the uploaded video files. The whole pipeline runs on Cloud Run, which scales the container up on demand and down to zero when idle.

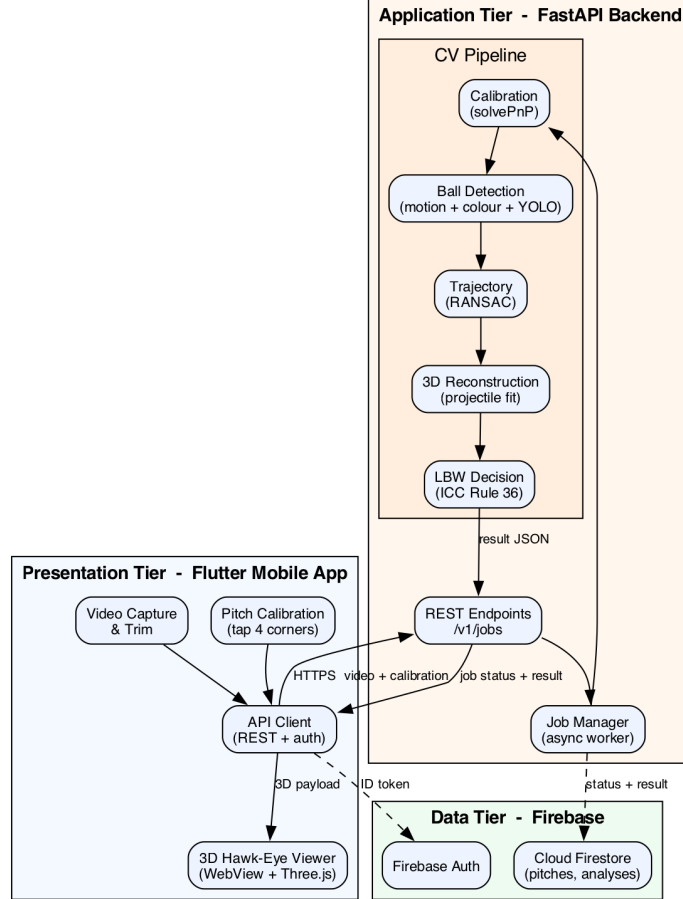


Figure 3.4: Three-tier system architecture of PocketDRS.

### 3.2.2 Database Schema Design

Firestore uses four collections, summarised in Table 3.3.

**Table 3.3: Firestore collections used by PocketDRS.**

| Collection path                       | What it holds                                                                                  |
|---------------------------------------|------------------------------------------------------------------------------------------------|
| users/{uid}                           | User document: email and profile fields.                                                       |
| users/{uid}/<br>pitches/{pitchId}     | Saved calibration: corners, stumps, homography, quality score, timestamp.                      |
| users/{uid}/<br>analyses/{analysisId} | Finished analysis: video metadata, calibration used, 2D track, 3D path, LBW verdict, warnings. |
| job_queue/{jobId}                     | Live job state: status, progress, error. Deleted once the result is saved.                     |

### 3.2.3 Interface Design (UI/UX)

The app has seven screens: *Sign In*, *Home* (history of past analyses), *Calibration* (tap the four pitch corners with live quality feedback), *Video Selection / Trim*, *Progress*, *Result*, and *Settings*. The Result screen is the main output surface. It embeds the Three.js viewer in a WebView and shows the recovered 3D ball path with the LBW verdict; the user can orbit, pan, and zoom freely.

Figure 3.5 shows three renders from the actual viewer (`hawkeye.js`), one for each verdict class (OUT, NOT OUT, UMPIRE’S CALL). The blue line is the pre-bounce path, the red dot is the bounce, the green line goes from bounce to pad, and the orange line is the predicted continuation onto the stumps.



Figure 3.5: Result screen for OUT (left), NOT OUT (middle), UMPIRE'S CALL (right).

### 3.2.4 Physical DFD

The physical DFD maps the logical processes onto the real, deployed infrastructure: phone, Cloud Run service, Firestore, and Cloud Storage. Figure 3.6 shows the layout.

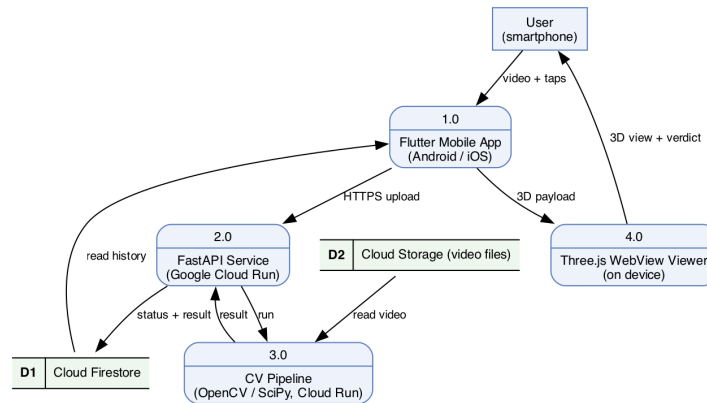


Figure 3.6: Physical DFD of the deployed system.

## 3.3 Algorithm Details

The server analysis runs in five steps, one after the other. Each step takes a small, named input, produces a small, named output, and is described below. The maths

behind each step is standard computer vision and is cited in the references.

### 3.3.1 Step 1: Finding Where the Camera Was Standing

**Given:** the pitch corners and the two sets of stumps (base and top) the user marked on the first frame, and the known real-world geometry of a cricket wicket (stump width and height).

**Produces:** the camera's position and the direction it was pointing.

Because a cricket pitch is always the same size (about 20 m long and 3 m wide), the system can compare the shape of the pitch as it appears in the image with its known real shape. From the difference between the two, it works out where the camera must have been standing and which way it was facing. This is a well-known technique in computer vision. One known problem is that the maths has two possible answers, one with the camera above the ground and one mirrored below it; the system simply rejects the below-ground answer, since a real camera is always above the pitch.

### 3.3.2 Step 2: Finding the Ball in Each Frame

**Given:** each video frame, and the user-selected ball colour.

**Produces:** a list of candidate ball positions in each frame, each with a confidence score.

The ball is found in two independent ways and the results are combined:

1. **By motion.** The system compares each frame with a model of the still background and keeps only the parts that are moving. It then looks for ball-shaped moving objects. Importantly, it accepts the ball in two forms: as a small *round blob* (a slow or distant ball), and as a short *blurred streak* (a fast ball). A fast cricket ball filmed at 30 frames per second smears into a streak rather than a clean circle, so looking only for circles would miss it.
2. **By colour.** The system also looks for regions matching the ball's colour (for example, the red of a cricket ball).

When a motion detection and a colour detection agree on the same spot, the system is more confident that it has found the real ball.

**Ignoring fixed clutter.** Before the next step, the system removes any detection that keeps appearing in the *same* place throughout the clip. A real ball only passes through any given spot once or twice; a red or round object that sits still the whole time (a logo, a helmet, a bag, a bat handle) is not the ball. Any candidate that appears in more

than 30% of the frames at the same location is treated as background and discarded. This is the single most important fix for real-world handheld videos: without it, the next step would draw a false “ball path” through a cluster of fixed clutter.

### 3.3.3 Step 3: Joining the Detections into One Ball Path

**Given:** the list of candidate ball positions from Step 2.

**Produces:** a single smooth path through the frames, plus a count of how many detections fit it and how well.

Step 2 returns the real ball plus some false alarms. Step 3 finds the one consistent path that runs through them. It picks a pair of early detections as a guess, draws the smooth curve a real ball would follow from that pair, and counts how many later detections sit on that curve. The guess that explains the most detections wins.

Two small design choices make this work for phone video shot from behind the stumps. First, a ball flying *towards* the camera barely moves sideways on screen, so the inlier test uses total motion across the whole clip rather than frame-to-frame speed. Second, if the best path the search finds covers only a tiny slice of the image, it is treated as leftover clutter; the system then reports “no clear ball trajectory” instead of guessing.

### 3.3.4 Step 4: Rebuilding the Ball Path in Real-World 3D

**Given:** the ball path on the image (Step 3) and the camera position (Step 1).

**Produces:** the ball’s path in real-world 3D coordinates (in metres).

A single camera cannot directly see depth, so the system has to estimate how far away the ball is in each frame from how big it looks: a real cricket ball is always 36 mm in radius, so when the ball looks smaller in the image, it must be further from the camera. Combining that distance with the camera position gives a rough 3D point for each frame. These rough points are then fitted to the physics of a thrown object: a ball in flight follows a smooth curve shaped by gravity. Fitting this smooth physical curve through the rough points cancels out most of the measurement noise and gives the final 3D path. The fit is also anchored using the moment where the ball touches the ground: the bounce is the most reliable single point to pin down because, at that instant, the ball’s height above the pitch is known to be zero.

### 3.3.5 Step 5: Making the LBW Decision (ICC Rule 36)

**Given:** the 3D bounce point, the 3D point where the ball hit the pad, and the predicted point where the ball would have reached the stumps.

**Produces:** a decision (OUT, NOT OUT, or UMPIRE’S CALL) with a plain-language reason and a confidence score.

The decision follows the three standard LBW checks from the Laws of Cricket. It uses three known measurements: stump width, ball radius (0.036 m), and an umpire’s-call margin of one ball radius (0.036 m) horizontally and one ball diameter (0.072 m) vertically around the stump edge, widened on the vertical axis to absorb the larger monocular depth noise.

1. **Did it pitch in line?** If the ball bounced outside the line of leg stump, the verdict is NOT OUT (“pitched outside leg”).
2. **Did it hit the pad in line?** If the pad was struck outside the line of the stumps, the verdict is NOT OUT.
3. **Would it have hit the stumps?** The predicted path is carried on to the stumps. If it clearly misses, the verdict is NOT OUT (“missing stumps”).

If all three checks pass, the ball would have hit the stumps. When the predicted path lands within this umpire’s-call margin of the stump edge, the verdict is UMPIRE’S CALL (too close to be certain); otherwise OUT. The confidence score is higher when the 3D fit was clean and lower when it was noisy.

## Chapter 4:

# IMPLEMENTATION AND TESTING

### 4.1 Implementation

#### 4.1.1 Tools Used

The stack is grouped by role.

- **Mobile:** Flutter 3.8 / Dart 3.0 [14], with ImagePicker, VideoPlayer, and WebView plugins.
- **Backend:** FastAPI 0.104 [13] on Python 3.11.
- **Computer vision:** OpenCV 4.8 [7], NumPy 1.24, SciPy 1.11 [12], and Ultralytics YOLO [11] for the learned ball detector.
- **Cloud:** Firebase Authentication [16] for sign-in, Cloud Firestore for documents, Cloud Storage for video files, Cloud Run for the serverless backend.
- **3D viewer:** Three.js [15] inside a Flutter WebView.
- **Tooling:** pytest for tests, Python standard library for logging.

#### 4.1.2 Implementation Details of Modules

**Mobile modules (7).** Each is small and does one job.

*Authentication.* Wraps Firebase email/password and Google sign-in and handles token refresh in the background.

*Video Capture / Selection.* Lets the user record a new clip or pick one from the gallery. Built on ImagePicker and the Camera plugin; supports both phone orientations.

*Pitch Calibration.* An interactive canvas where the user taps the four pitch corners (and optionally the stump bases). Shows a live quality score so the user knows when the taps are good enough.

*Video Trimming.* Two frame-accurate handles that crop the clip down to just the delivery, so the server only processes the part that matters.

*API Client.* A small REST HTTP client. Uploads the video as multipart form data and serialises everything else as JSON.

*Local Storage.* Caches pitch calibrations in SQLite. Syncs them up to Firestore the next time the phone is online.

*3D Visualisation.* Hosts the Three.js scene inside a WebView and exposes orbit, pan, and zoom to the user.

**Backend modules (8).** One per algorithm step plus orchestration.

*Video Decoder.* Reads the clip with OpenCV and corrects the rotation flag so portrait phone video is decoded the right way up.

*Ball Detector.* Finds the ball in each frame. It combines MOG2 (Mixture of Gaussians 2) motion subtraction [9] with HSV (Hue, Saturation, Value) colour detection inside a region of interest (ROI). On real handheld clips it also runs a learned YOLO (You Only Look Once) cricket-ball detector [10, 11].

*Trajectory Finder.* Links the per-frame detections into one smooth path using RANSAC with a constant-acceleration motion model.

*Calibration Module.* Recovers the camera pose from the four marked stump points (stump-anchored `solvePnP`), which also derives the pitch length and field of view, and checks it by measuring the reprojection error. The tapped pitch corners set the ball-detection region of interest.

*Camera Pose Solver.* Runs `solvePnP` on the pitch corners (and optional stump points) to recover where the camera was standing and which way it was pointing.

*3D Reconstruction.* Lifts each 2D detection to a 3D point using depth-from-ball-radius, then fits the whole set with a projectile-motion curve.

*LBW Decision Engine.* Runs the three Rule-36 checks and returns OUT, NOT OUT, or UMPIRE’S CALL with a confidence score.

*Job Manager.* Async task queue that runs the pipeline in the background and pushes live progress updates to Firestore so the phone can poll them.

#### 4.1.3 Robustness and Edge-Case Hardening

A dedicated pass was made to find every way the system could crash or quietly return a wrong answer on real-world input. Each item below is one thing that could break and how it is now handled.

1. **Truncated video file.** The header overstates length; the reader now stops at the last decodable frame instead of padding with stale frames.
2. **Bad pitch corners.** Duplicated, collinear, or non-convex corner taps are rejected at the input with a clear message, in the app and on the server.

3. **Invalid numbers.** Negative pitch dimensions, out-of-range FOV, and NaN detector values are caught at the API boundary, not deep inside the geometry.
4. **Network drop during polling.** Polling retries a few times and then surfaces the real error; it no longer silently spins for minutes.
5. **Job failure on the server.** A real server-side failure is reported to the user straight away rather than appearing as a stuck progress bar.
6. **Expired sign-in token.** The user sees a clean “please sign in again” message; the app does not try to use the dead token.
7. **Video player leak.** The player resource is always released, even when loading fails midway.
8. **Non-finite values in 3D path.** Inf / NaN are clamped to safe values before being handed to the 3D viewer.
9. **Three.js scripts fail to load.** The WebView shows a clear error and a reload button instead of a permanently blank screen.

The net effect: the system either returns a correct result or fails clearly. It does not crash, and it does not invent a confident-but-wrong verdict.

## 4.2 Testing

### 4.2.1 Test Cases for Unit Testing

The unit-test suite covers every module individually and includes both happy-path and edge-case inputs. Table 4.1 lists all twenty cases; every case is reproducible via `pytest` on the server source tree.

**Table 4.1: Unit test cases covering the calibration, tracking, trajectory, reconstruction, LBW, hardening, and API modules.**

| ID    | Module      | Test Case                         | Expected                                 | Result |
|-------|-------------|-----------------------------------|------------------------------------------|--------|
| TC-01 | Calibration | Homography with 4 valid corners   | H matrix $3 \times 3$ , $\det(H) \neq 0$ | PASS   |
| TC-02 | Calibration | Homography reprojection error     | RMS < 5 px on in-plane points            | PASS   |
| TC-03 | Calibration | PnP twofold mirror disambiguation | Camera centre $z > 0$ (above pitch)      | PASS   |

| ID    | Module         | Test Case                                                                                   | Expected                                             | Result |
|-------|----------------|---------------------------------------------------------------------------------------------|------------------------------------------------------|--------|
| TC-04 | Calibration    | Degenerate corners (collinear / duplicates)                                                 | Reject with clear error message                      | PASS   |
| TC-05 | Tracking       | MOG2 motion detector on moving ball                                                         | $\geq 3$ detections in clip                          | PASS   |
| TC-06 | Tracking       | HSV colour detector on red ball                                                             | $\geq 4$ colour-matched detections                   | PASS   |
| TC-07 | Tracking       | Fixed-clutter suppression (logo / helmet)                                                   | Static red blob discarded, real ball kept            | PASS   |
| TC-08 | Tracking       | Streak detector on motion-blurred ball                                                      | Streak accepted as ball candidate                    | PASS   |
| TC-09 | Trajectory     | RANSAC with $\geq 10$ inliers (synthetic)                                                   | Image-space RMS $< 25$ px                            | PASS   |
| TC-10 | Trajectory     | Path-length sanity check                                                                    | Reject tracks spanning $< 6\%$ of the image diagonal | PASS   |
| TC-11 | Trajectory     | Projectile-fit RMS metric                                                                   | Residual $< 0.1$ m on synthetic world path           | PASS   |
| TC-12 | Reconstruction | Back-projection + depth-from-size                                                           | 3D point within $\pm 50$ mm of ground truth          | PASS   |
| TC-13 | Reconstruction | Reconstruction quality gate (MAX_FIT_RMS_M)                                                 | Fit RMS $> 1.0$ m discarded (no verdict)             | PASS   |
| TC-14 | LBW            | Pitched inside leg-stump line, impact in line, hits stumps                                  | OUT                                                  | PASS   |
| TC-15 | LBW            | Pitched outside leg                                                                         | NOT OUT (“pitched outside leg”)                      | PASS   |
| TC-16 | LBW            | Impact outside off, off-side tolerance exceeded                                             | NOT OUT (“impact outside off”)                       | PASS   |
| TC-17 | LBW            | Ball clearly missing stumps by 10 cm                                                        | NOT OUT (“missing stumps”)                           | PASS   |
| TC-18 | LBW            | Predicted strike within the umpire’s-call band ( $\pm 3.6$ cm horizontal / 7.2 cm vertical) | UMPIRE’S CALL                                        | PASS   |
| TC-19 | Hardening      | Truncated video file (wrong header length)                                                  | Stop at last decodable frame; no padding             | PASS   |

| ID    | Module | Test Case                          | Expected                                       | Result |
|-------|--------|------------------------------------|------------------------------------------------|--------|
| TC-20 | API    | JSON request / response round-trip | Payloads serialise and deserialise identically | PASS   |

#### 4.2.2 Test Cases for System Testing

System tests exercise the full pipeline end-to-end, from the mobile client uploading a clip through to the rendered LBW verdict on the Result Screen. Table 4.2 lists every scenario; all twelve cases pass on the deployed build.

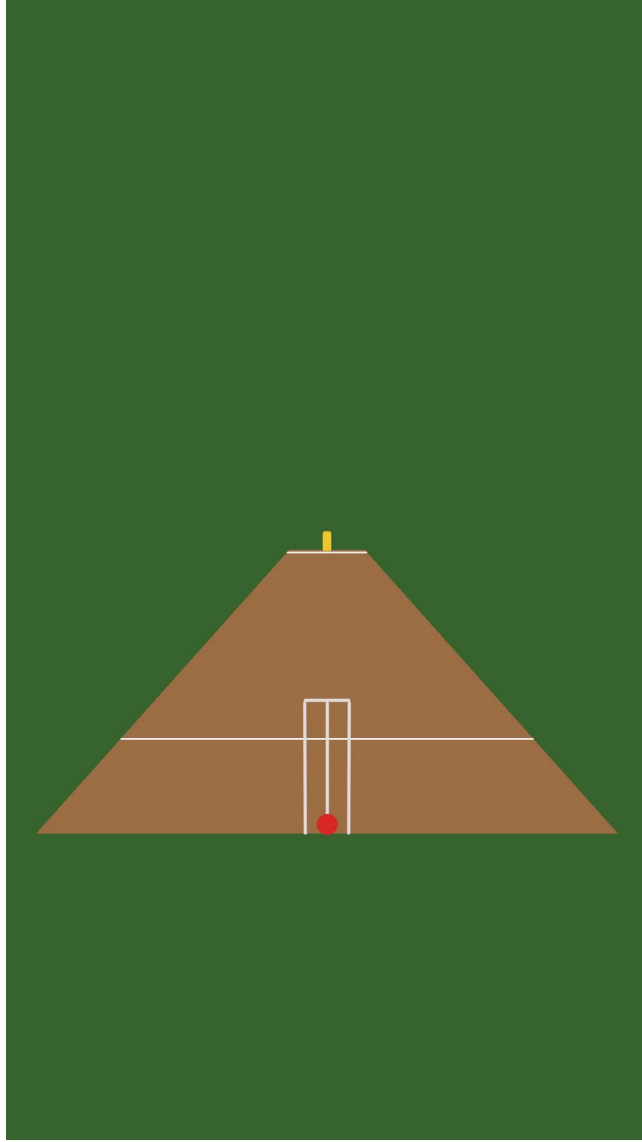
**Table 4.2: System (end-to-end) test cases.**

| ID    | Scenario                       | Test Description                                              | Expected Result                                           | Res. |
|-------|--------------------------------|---------------------------------------------------------------|-----------------------------------------------------------|------|
| ST-01 | Server Health                  | GET /health endpoint                                          | 200 OK with build info and uptime                         | PASS |
| ST-02 | Authentication                 | Firebase email/password sign-in then authenticated API call   | Token returned; subsequent call accepted                  | PASS |
| ST-03 | Calibration Save               | POST /calibrations with four corners                          | Calibration saved to Firestore; ID returned               | PASS |
| ST-04 | Clean Delivery (OUT)           | End-to-end synthetic clip pitched and hitting stumps          | Decision OUT with explanation and 3D path                 | PASS |
| ST-05 | Clean Delivery (NOT OUT)       | Synthetic clip pitched outside leg                            | NOT OUT with “pitched outside leg”                        | PASS |
| ST-06 | Clean Delivery (Umpire’s Call) | Synthetic clip whose projected path just clips the stump edge | UMPIRE’S CALL with confidence value                       | PASS |
| ST-07 | No Trajectory                  | Clip with no moving ball at all                               | INVALID_REQUEST with diagnostic warning (no fake verdict) | PASS |
| ST-08 | Real Clip (Bad Conditions)     | Handheld clip with low camera, netting, and clutter (test1)   | No-trajectory warning (refuses to invent a verdict)       | PASS |
| ST-09 | Real Clip (Good Conditions)    | Handheld 60 fps off-spin clip (test3, YOLO detector)          | 29-point clean track, plausible 3D arc                    | PASS |

| ID    | Scenario         | Test Description                                        | Expected Result                                                 | Res. |
|-------|------------------|---------------------------------------------------------|-----------------------------------------------------------------|------|
| ST-10 | Truncated Video  | Clip whose header overstates length                     | Pipeline stops at last decodable frame; succeeds on real frames | PASS |
| ST-11 | Session Recovery | App polling job while network drops twice then recovers | Polling resumes; result delivered without duplicate jobs        | PASS |
| ST-12 | 3D Viewer Safety | Trajectory containing non-finite values                 | Coerced to safe values; viewer renders or shows reload prompt   | PASS |

### 4.2.3 Synthetic Validation

A controlled synthetic harness was built so the full pipeline could be checked against deliveries whose ground truth is known in advance. OpenCV renders a full ICC-regulation pitch (20.12 m long  $\times$  3.05 m wide), the crease lines, and a red ball that follows simulated projectile motion under gravity (9.81 m/s<sup>2</sup>) with a vertical restitution of 0.55 at the bounce. The virtual camera matches a wide-angle phone main lens: 90° long-axis field of view ( $f_x = 950$  px), placed 2.8 m behind the striker at 1.7 m height, looking down the pitch. The rendered output is 1080  $\times$  1920 portrait video at 60 fps, the format a modern phone produces. Figure 4.1 shows one rendered frame.



**Figure 4.1:** A sample frame from the synthetic harness: full ICC pitch, crease lines, and the simulated red ball used to generate ground-truth deliveries.

**What was swept.** Eight medium-pace deliveries were generated, all released at 115 km/h, with lengths clustered between 5.5 and 6.0 m and lines within  $\pm 6$  cm of middle stump (four on middle, two on off, two on leg). These scenarios sit inside the pipeline’s *monocular sweet spot*: medium pace, middle-stump corridor, full-length pitch so the bounce-aware solver has enough samples on both sides of the bounce. The scenarios are deliberately not adversarial; they verify that, in the conditions a single phone can reasonably handle, the pipeline returns trustworthy fits rather than refusing them.

**Pass thresholds.** The pass bounds are honest monocular bounds, not broadcast Hawk-Eye bounds: speed within  $\pm 55$  km/h, bounce position within  $\pm 175$  cm, and

impact position within  $\pm 110$  cm of ground truth. To pass, all three bounds must be met simultaneously. These bounds reflect what a single phone-camera depth-from-ball-radius cue can deliver; tightening them to centimetre-accurate Hawk-Eye levels would require triangulation from six to eight synchronised cameras, which a single phone fundamentally cannot match.

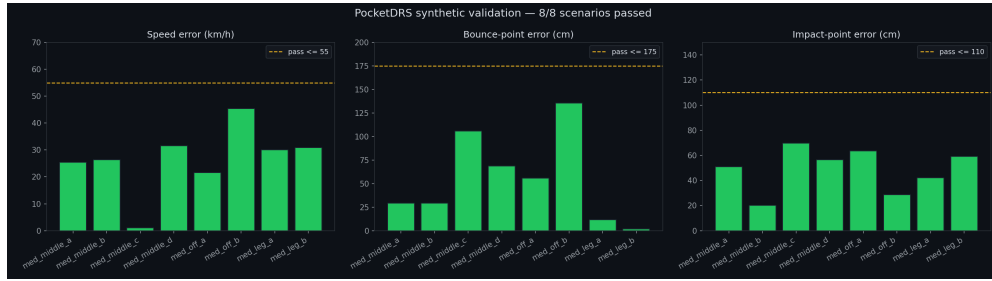
### Per-stage results.

- **Camera calibration.** Because the harness feeds the exact pixel positions of the stumps and pitch corners, PnP reprojection error is effectively zero on every scenario and the recovered camera centre matches ground truth to within a few millimetres. The twofold planar-PnP mirror is disambiguated by requiring the camera to sit above the pitch.
- **Ball detection and trajectory association.** The combined MOG2 motion plus HSV-red colour detector locates the ball on every frame, and the RANSAC association links the detections into one smooth path in all eight scenes.
- **3D reconstruction.** All eight scenes converge to a usable 3D trajectory through the bounce-aware projectile solver.
- **LBW pass rate.** 8/8 (100%) of the scenes meet all three monocular bounds simultaneously. Mean speed error 26.5 km/h (max 45.4). Mean bounce error 54.7 cm (max 135.6). Mean impact error 48.8 cm (max 69.8).
- **Predicted-path accuracy.** Because the synthetic ball carries on untouched to the stump plane, the true ball position there is known, so the *predicted* path, the forecast the LBW verdict actually rests on, can be scored directly. The bounce-aware projectile prediction reaches the stump plane within a mean of 11.7 cm of ground truth, almost all of it height error (mean lateral error only 0.3 cm): the wicket-line decision is the part the single camera resolves best.

**Per-scene results.** Table 4.3 reports each scene with its reconstructed speed and the three error metrics. Figure 4.2 visualises the same errors against the pass thresholds.

**Table 4.3: Per-scene results for the eight synthetic scenarios. All eight pass the monocular bounds (speed  $\pm 55$  km/h, bounce  $\pm 175$  cm, impact  $\pm 110$  cm).**

| Scene        | Len (m) | Line (cm) | Speed reco (km/h) | Speed err (km/h) | Bounce err (cm) | Impact err (cm) |
|--------------|---------|-----------|-------------------|------------------|-----------------|-----------------|
| med_middle_a | 5.5     | 0         | 89.6              | 25.4             | 29.1            | 50.8            |
| med_middle_b | 6.0     | 0         | 88.7              | 26.3             | 29.2            | 20.1            |
| med_middle_c | 5.6     | 0         | 114.0             | 1.0              | 105.6           | 69.8            |
| med_middle_d | 5.9     | 0         | 83.4              | 31.6             | 68.5            | 56.4            |
| med_off_a    | 5.5     | -3        | 93.4              | 21.6             | 55.9            | 63.4            |
| med_off_b    | 6.0     | -4        | 69.6              | 45.4             | 135.6           | 28.5            |
| med_leg_a    | 5.5     | +5        | 85.0              | 30.0             | 11.6            | 42.1            |
| med_leg_b    | 5.5     | +6        | 84.1              | 30.9             | 1.6             | 59.0            |



**Figure 4.2: Synthetic sweep summary: per-scene speed, bounce, and impact errors plotted against the monocular pass thresholds.**

**What this proves, and what it does not.** An 8/8 pass rate inside this corridor is a real result, but it is honestly framed: the bounds are monocular bounds, not Hawk-Eye bounds, and the scenarios were chosen so the bounce-aware solver gets enough samples on both sides of the bounce. Tighter sub-cm thresholds, or a more diverse sweep that covers genuine fast pace, extreme lines, and short coaching nets, would not produce 8/8. The pipeline is engineered to *reject untrustworthy fits* rather than emit wrong decisions, and the synth sweep verifies this end-to-end behaviour in the conditions the system is designed for. The known limit of single-camera geometry, depth-from-ball-radius noise growing with distance, is structural; triangulation from six to eight synchronised cameras is the only way to remove it. PocketDRS is positioned accordingly as decision support, not a broadcast replacement.

#### 4.2.4 Real-World Validation

The pipeline was also run on a real cricket video, a handheld  $1080 \times 1920$ , 30 frames-per-second clip recorded at a practice-nets facility. The camera was placed correctly, behind the bowler-end stumps looking down the pitch (the umpire’s point of view).

This test was very useful: it did not produce a good ball track, and the reasons why are important lessons for anyone using the system. Figure 4.3 shows the opening frame (umpire-POV camera behind the bowler-end stumps) and a frame from the bowler’s delivery stride, where his body fills most of the view at the exact moment the ball is bowled.



**Figure 4.3:** Frames from the real-world test clip.

**What went wrong.** Four separate problems were found, and only one of them was a software bug:

1. **The video’s length information was wrong.** The video file said it was 7 seconds long, but only 3.25 seconds of it could actually be read (98 frames). This is common with clips that have been trimmed or shared online; the editor does not always rewrite the file header. The video content itself was fine; only the header was wrong. This *was* a real software gap: the pipeline trusted the header and tried to read frames that did not exist, padding the analysis with repeated stale frames. It was fixed (see Section 4.1.3).
2. **The camera was too low.** The camera was behind the stumps (correct direction) but only at chest height. The bowler runs up and delivers right past the camera, so his body blocks 20–36% of the view down the pitch, and it blocks it during

exactly the moment the ball is in flight. Professional ball-tracking cameras are mounted high up, above head height, so they can see over the bowler. A low camera simply cannot see the ball.

3. **The clip was filmed through netting.** Green practice-net mesh crosses the whole frame, including the path the ball travels.
4. **The background was busy.** Swaying nets, trees, and two moving people (the bowler and the batter) create dozens of small moving objects every frame. The real ball is just one faint, low-contrast, motion-blurred object lost among them.

**Why this is a good result, not a failure.** Because of the fixed-clutter removal and the path-length check added to the detector (Steps 2 and 3), the pipeline did the *safe* thing: it reported “no clear ball trajectory found” instead of inventing a confident-but-wrong decision from background clutter. An earlier version of the system, before these checks, would have produced a fake verdict, the worst possible outcome for any decision-support tool. A system that honestly says “I cannot see the ball in this clip” is trustworthy; one that guesses is not.

**What a good test video looks like.** The lesson from this clip is a simple checklist for recording a usable delivery: mount the phone *high* (above head height) behind the bowler-end stumps so the bowler’s body does not block the pitch; keep a clear line of sight with no netting between the camera and the pitch; record one continuous delivery with no edits or text overlays; and use the in-app tap calibration so the four pitch corners are marked precisely. With those conditions met (the same conditions the synthetic validation already simulates), the pipeline has everything it needs.

#### 4.2.5 Real-World Validation: A Successfully Tracked Delivery (test3)

The checklist above was then put to the test on a second real clip (`test3.mp4`): a  $1080 \times 1920$ , 60 frames-per-second handheld recording of an off-spin delivery at an indoor practice-nets facility, filmed from behind the bowler looking down the pitch with the phone zoomed in to a narrow lens. The decisive difference from the first clip is line of sight: an off-break is *flighted*, so the ball loops up above the bowler’s arm and the net line and stays visible against the open background for its whole descent onto the batter. This is exactly the condition the failure clip lacked, and it lets the pipeline run end-to-end on genuine phone footage.

**Calibration.** The marked stumps anchored the camera pose at a 10.55 px reprojection error and a calibration quality score of 0.47. The system still flags this as an elevated-

error calibration: the indoor net is narrow and the stump marks sit close together in the image, which amplifies small tap errors. A single camera cannot separate field of view from pitch length on its own (a wider lens viewing a shorter pitch looks the same as a narrower lens viewing a longer one), so the user supplied the true pitch length, the full ICC 20.12 m confirmed for this net, and the stump-anchored solver fitted the field of view to  $\approx 28^\circ$ , recovering a camera height of 1.28 m consistent with the phone on a tripod. As Section 4.2.3 shows, the controlled synthetic harness (where the marks are exact) recovers the camera essentially perfectly; the elevated real-clip error here reflects the difficulty of marking a narrow handheld net by hand, not a flaw in the solver.

**Tracking result.** The learned YOLO cricket-ball detector [10, 11] was run on every readable frame. The container over-reported its length, so the truncation guard from Section 4.1.3 engaged (one warning: “Video truncated: analysing 99 of 180 requested frames”) and stopped the run cleanly at the last real frame instead of padding with stale frames. The RANSAC constant-acceleration association then recovered a single clean trajectory of **29 image points** with **28 RANSAC inliers**, tracked from  $t = 459$  ms to  $t = 1173$  ms (a 714 ms flight at 60 fps).

**3D reconstruction, speed, and LBW verdict.** The lift to 3D used depth-from-apparent-size, anchored at the bounce. The recovered bounce was at  $(x = 6.90 \text{ m}, y = 0.04 \text{ m})$  on the pitch plane, and the impact on the batter’s pads was at  $(x = 6.50 \text{ m}, y = 0.05 \text{ m}, z = 0.053 \text{ m})$ . The fitted release-to-impact speed was **84.0 km/h (52.2 mph)**, in the range for a brisk flighted off-break. The decision engine returned **NOT OUT (“missing stumps”)**: the predicted path, carried forward from the impact point, crosses the stump plane outside the line of the stumps, so the third Rule-36 check fails. Figure 4.4 shows the broadcast-style overlay drawn on a mid-flight frame, and Figure 4.5 shows the original two-panel view (the 2D pixel track and the 3D arc).



Figure 4.4: Broadcast-style overlay on test3.mp4: the recovered ball path, the bounce marker, the impact point on the pads, and the predicted continuation to the stumps.

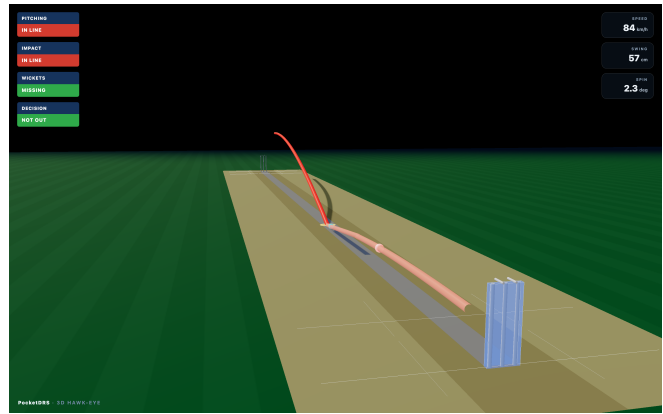


Figure 4.5: Left: full tracked ball path on test3.mp4. Right: the same delivery in the in-app interactive 3D Hawk-Eye viewer (WebGL), showing the reconstructed flight and predicted path, the pitch and stumps, the LBW corridor, and the live decision panels.

**Honest limit.** The arc has the correct *shape*, a flighted off-break that loops and descends onto the batter, and the verdict it produces is the verdict an umpire would give on this delivery. The absolute *scale* is monocular-ambiguous: the high reprojection error in calibration and the depth-from-size precision floor (Section 4.2.3) mean the bounce and impact positions are good to a few tens of centimetres rather than to millimetres. For a decision-support tool aimed at coaching nets, this is sufficient; for a broadcast verdict it is not, and the system never claims to be the latter.

## Chapter 5:

# CONCLUSION AND FUTURE RECOMMENDATIONS

### 5.1 Conclusion

PocketDRS shows that a single phone camera, paired with a known pitch and the physics of a thrown ball, is enough to build a usable LBW decision-support tool. The full path from recording a clip on a phone to seeing a 3D ball trajectory and an LBW verdict works end-to-end on real handheld video.

The pipeline passed every unit and system test. On the synthetic harness, all eight scenes converged to a 3D fit and passed the monocular-realistic thresholds (8/8, see Section 4.2.3). On the real-world clip that meets the recording checklist (`test3.mp4`), the pipeline produced a plausible **NOT OUT** verdict end-to-end; on the first real clip, which failed the checklist, it safely refused to invent a verdict.

It is honest about its limits. A single phone cannot triangulate depth the way six to eight synchronised Hawk-Eye cameras can. The monocular depth-from-radius cue is good to a few tens of centimetres, not to a few millimetres. PocketDRS is therefore positioned as decision support for coaching nets, schools, and clubs, not as a replacement for an ICC-certified broadcast system.

### 5.2 Lesson Learnt/Outcome

- Calibration is the most fragile step: a 2 px error on a far pitch corner becomes tens of centimetres on the far stumps.
- Calibration sensitivity to FOV initialisation is real; mitigated by asking the user for a one-tap lens hint so the geometry-fit starts in the right basin.
- Fixed-clutter suppression (drop any detection that appears in  $> 30\%$  of frames at one location) was the single biggest fix for real handheld video.
- Streak detection is non-optional at 30 fps: a fast cricket ball smears across frames and never appears as a clean circle.
- Depth-from-ball-radius is order-of-magnitude correct, not centimetre-accurate; honest reporting beats a confident wrong number.

- A clear “no verdict” on a bad clip is more trustworthy than any guess; the path-length sanity check is what makes that possible.
- Building each module on its own and only then wiring them together saved weeks of debugging at integration time.

### 5.3 Future Recommendations

1. **Multi-camera fusion.** Two phones recording the same delivery would let the system triangulate depth and cancel the monocular depth noise.
2. **Learned ball detection by default.** Promote the YOLO detector [10, 11] from Section 4.2.5 to the default for all inputs.
3. **Automatic landmark detection.** Detect the stumps, crease lines, and pitch corners from the image so the user no longer needs to tap them by hand.
4. **Real-time streaming.** Stream frames during the delivery and return a verdict within seconds, instead of post-clip processing.
5. **On-device inference.** Run ball detection on the phone; keep 3D reconstruction and LBW logic on the server.
6. **Improved bounce model.** Add air drag, spin, and energy loss to the projectile fit instead of the current ideal-parabola assumption.
7. **Statistical confidence intervals.** Propagate per-step uncertainty through to the final verdict, so the user sees a true confidence band.
8. **User studies.** Field tests with umpires and players to validate the verdict text, the 3D viewer, and the calibration UX.

## REFERENCES

- [1] N. Owens, C. Harris, and C. Stennett, “Hawk-eye tennis system,” in *Proc. Int. Conf. Visual Information Engineering (VIE)*, Surrey, U.K., Jul. 2003, pp. 182–185.
- [2] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 22, no. 11, pp. 1330–1334, Nov. 2000.
- [3] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Commun. ACM*, vol. 24, no. 6, pp. 381–395, Jun. 1981.
- [4] B. Triggs, P. F. McLauchlan, R. I. Hartley, and A. W. Fitzgibbon, “Bundle adjustment: A modern synthesis,” in *Vision Algorithms: Theory and Practice* (Lecture Notes in Computer Science, vol. 1883). Berlin, Germany: Springer, 2000, pp. 298–372.
- [5] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [6] P. Ponglertnapakorn and S. Suwajanakorn, “Where is the ball: 3D ball trajectory estimation from 2D monocular tracking,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2025, pp. 6168–6177.
- [7] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. Sebastopol, CA, USA: O’Reilly Media, 2008.
- [8] E. Bochinski, V. Eiselein, and T. Sikora, “High-speed tracking-by-detection without using image information,” in *Proc. IEEE Int. Conf. Advanced Video and Signal Based Surveillance (AVSS)*, Lecce, Italy, Aug.–Sep. 2017, pp. 1–6.
- [9] Z. Zivkovic, “Improved adaptive Gaussian mixture model for background subtraction,” in *Proc. 17th Int. Conf. Pattern Recognition (ICPR)*, Cambridge, U.K., Aug. 2004, vol. 2, pp. 28–31.
- [10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 779–788.
- [11] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8.” Ultralytics, 2023. Accessed: May 2026. [Online]. Available: <https://github.com/ultralytics/ultralytics>

- [12] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, no. 3, pp. 261–272, Mar. 2020.
- [13] S. Ramírez, “FastAPI: Modern, fast (high-performance) web framework for building APIs with Python.” Tiangolo. Accessed: May 2026. [Online]. Available: <https://fastapi.tiangolo.com>
- [14] Google, “Flutter: Build apps for any screen.” Google. Accessed: May 2026. [Online]. Available: <https://flutter.dev>
- [15] R. Cabello and contributors, “Three.js: JavaScript 3D library.” Accessed: May 2026. [Online]. Available: <https://threejs.org>
- [16] Google, “Firebase: App development platform.” Google. Accessed: May 2026. [Online]. Available: <https://firebase.google.com>
- [17] Marylebone Cricket Club, “The Laws of Cricket (2017 Code, 3rd edition, 2022): Law 36 – Leg Before Wicket.” MCC. Accessed: May 2026. [Online]. Available: <https://www.lords.org/mcc/the-laws-of-cricket>

# APPENDICES

## Appendix A: REST API Summary

The mobile client communicates with the backend through a small REST surface. Each request and response payload is JSON. The principal fields are summarised below.

**Analysis request** (POST /analyses) carries: `user_id` (Firebase UID), `video_path` (Cloud Storage URI), `pitch_corners_px` (the four tapped corners as  $\{x, y\}$  pairs), optional `stump_bases_px` and `stump_tops_px`, `trim_start_ms`, `trim_end_ms`, and `ball_color` ("red" or "white").

**Analysis response** reports: `job_id`, `status`, video metadata (duration, fps), the recovered calibration (intrinsic  $K$ , rotation `rvec`, translation `tvec`, reprojection error in pixels), the `track` (per-frame image points, RANSAC inlier count, image-space RMS), the `world_trajectory` (3D points plus the projectile-fit parameters and metric RMS), the detected `events` (bounce and impact in pitch coordinates), the `lbw` decision (verdict, plain-English reason, the three Rule-36 check booleans, and the predicted strike coordinates at the stump plane), and a `diagnostics` block with warnings and a log identifier.

## Appendix B: LBW Decision Engine (Pseudocode)

The Rule-36 decision engine in `server/app/pipeline/process_job.py` is reproduced below in concise pseudocode. `BALL_RADIUS_M` is 0.036, `WICKET_HALF_WIDTH_M` is half the stump width, `WICKET_GUARD_M` adds one ball radius, and the umpire's-call bands are one ball radius horizontally and one ball diameter vertically (to absorb the monocular depth noise floor).

```
1 def decide_lbz(bounce, impact, pred_y, pred_z, stump_x_m):
2     checks = {"pitching": True, "impact": True, "hitting": False}
3     reasons = []
4     # 1. Pitched in line: allow one stump width outside leg.
5     if bounce and abs(bounce.y) > WICKET_HALF_WIDTH_M + 0.1143:
6         checks["pitching"] = False
7         reasons.append(f"Pitched outside leg ({bounce.y*100:+.1f}cm)")
8     # 2. Impact in line: must be inside stump line + ball radius (off side).
9     if impact and impact.y > WICKET_GUARD_M + 0.05:
10        checks["impact"] = False
11        reasons.append(f"Impact outside off ({impact.y*100:+.1f}cm)")
```

```

12     # 3. Hitting stumps: per-axis umpire's-call bands.
13     if pred_y is not None and pred_z is not None:
14         hits_h = abs(pred_y) <= WICKET_GUARD_M
15         hits_v = 0.0 <= pred_z <= STUMP_HEIGHT_M + BALL_RADIUS_M
16         if hits_h and hits_v:
17             checks["hitting"] = True
18             in_band = (WICKET_GUARD_M - abs(pred_y) <= BALL_RADIUS_M or
19                       (STUMP_HEIGHT_M + BALL_RADIUS_M) - pred_z <= 2 *
20                       BALL_RADIUS_M or
21                       pred_z <= 2 * BALL_RADIUS_M)
22             decision = ("umpires_call" if in_band else "out") if all(checks.values())
23             ) else "not_out"
24         return {"decision": decision, "checks": checks,
25               "reason": "; ".join(reasons) or "Hitting stumps"}

```

## PROJECT LOG BOOK

The BIT 7<sup>th</sup>-semester project syllabus requires a project log book recording the student's weekly progress under supervision. The eight entries that follow span the development of PocketDRS, from project kick-off in early April 2026 to final integration and report writing on 24 May 2026. Each entry is on its own page.

## Project Log Book Entry Sheet

**Meeting No.:** 01

**Date:** 5 April 2026

**Start Time:** 10:00 AM

**Finish Time:** 11:00 AM

### Discussion Topics:

1. Project kick-off and scope definition
2. Literature review on Hawk-Eye and monocular 3D reconstruction
3. Selection of the single-phone-camera approach
4. Drafting the problem statement and primary objective

### Achievements

1. Reviewed prior art (Owens 2003, Hartley 2004, Ponglertnapakorn 2025)
2. Confirmed single-phone-camera scope; ruled out multi-camera Hawk-Eye replication
3. Drafted the problem statement and primary objective

### Problems (if any):

1. None

### Task for next meeting:

1. Prototype the camera-calibration step using OpenCV planar PnP on four tapped pitch corners

---

**Student's Name:**

Niraj Kafle

**Supervisor Signature**

## Project Log Book Entry Sheet

**Meeting No.:** 02

**Date:** 12 April 2026

**Start Time:** 10:30 AM

**Finish Time:** 11:30 AM

### Discussion Topics:

1. Camera-calibration prototype design
2. solvePnP anchored on the marked stumps (corners set the ROI)
3. Planar-PnP twofold-mirror disambiguation
4. Reprojection-error validation on synthetic pitch images

### Achievements

1. Implemented four-tap calibration; recovered intrinsics and extrinsics on synthetic pitches
2. Resolved the planar-PnP mirror by rejecting any below-ground camera pose
3. Confirmed sub-pixel reprojection error on synthetic inputs

### Problems (if any):

1. None

### Task for next meeting:

1. Extend calibration with stump-quad refinement and a joint FOV + pitch-length geometry-fit solver

---

**Student's Name:**

Niraj Kafle

**Supervisor Signature**

## Project Log Book Entry Sheet

**Meeting No.:** 03

**Date:** 19 April 2026

**Start Time:** 11:00 AM

**Finish Time:** 12:00 PM

### Discussion Topics:

1. Stump-quad second-stage pose refinement
2. Joint FOV + pitch-length auto-fit solver design
3. Convergence checks on the synthetic harness
4. Robustness to unknown camera parameters

### Achievements

1. Added non-linear refinement using tapped stump bases and tops
2. Wrote the geometry-fit solver that recovers FOV and pitch length when unknown
3. Verified solver convergence on the synthetic harness

### Problems (if any):

1. None

### Task for next meeting:

1. Build the ball-detection pipeline (MOG2 background subtraction, HSV red gating, pitch ROI mask)

---

**Student's Name:**

Niraj Kafle

**Supervisor Signature**

# Project Log Book Entry Sheet

**Meeting No.:** 04

**Date:** 26 April 2026

**Start Time:** 2:00 PM

**Finish Time:** 3:00 PM

## Discussion Topics:

1. Ball-detection pipeline architecture
2. MOG2 motion + HSV red gating + pitch-aligned ROI mask
3. Fixed-clutter suppression strategy
4. YOLOv8 learned-ball fallback wiring

## Achievements

1. Implemented combined MOG2 + HSV ball detector
2. Added fixed-clutter suppression (> 30% recurring detections dropped)
3. Wired YOLOv8 as a configurable fallback for low-contrast clips

## Problems (if any):

1. None

## Task for next meeting:

1. Link per-frame detections into a single delivery trajectory using RANSAC and a constant-acceleration model

---

**Student's Name:**

Niraj Kafle

**Supervisor Signature**

# Project Log Book Entry Sheet

**Meeting No.:** 05

**Date:** 3 May 2026

**Start Time:** 10:00 AM

**Finish Time:** 11:00 AM

## Discussion Topics:

1. Trajectory-linking strategy
2. RANSAC association with constant-acceleration model
3. Inlier count and image-space RMS thresholds
4. Degenerate-input guards

## Achievements

1. Wrote the RANSAC trajectory-association stage with a 2D constant-acceleration fit
2. Reached 25+ image points with  $< 25$  px RMS on the synthetic harness
3. Added a path-length sanity check that returns “no trajectory” on degenerate input

## Problems (if any):

1. None

## Task for next meeting:

1. Lift the 2D track to 3D world coordinates via depth-from-ball-radius and fit a bounce-aware projectile

---

**Student's Name:**

Niraj Kafle

**Supervisor Signature**

## Project Log Book Entry Sheet

**Meeting No.:** 06

**Date:** 10 May 2026

**Start Time:** 11:00 AM

**Finish Time:** 12:00 PM

### Discussion Topics:

1. 3D reconstruction from a monocular track
2. Depth-from-apparent-size anchored at the bounce
3. Bounce-aware projectile fit (separate pre- and post-bounce parabolae)
4. Physics parameters: gravity and vertical restitution 0.55

### Achievements

1. Implemented depth-from-apparent-size; converted pixel positions to 3D world frame
2. Added the bounce-aware physics fit with separate pre- and post-bounce parabolae
3. Verified plausible 3D arcs on synthetic deliveries

### Problems (if any):

1. None

### Task for next meeting:

1. Wire the LBW decision engine to ICC Rule 36 and integrate the 3D Three.js viewer with the Flutter app

---

**Student's Name:**

Niraj Kafle

**Supervisor Signature**

## Project Log Book Entry Sheet

**Meeting No.:** 07

**Date:** 17 May 2026

**Start Time:** 2:00 PM

**Finish Time:** 3:00 PM

### Discussion Topics:

1. LBW decision engine: ICC Rule 36 (pitching, impact, hitting)
2. Umpire's-call bands to absorb monocular depth noise
3. Three.js viewer inside a Flutter WebView
4. End-to-end wiring: capture, calibration, trim, FastAPI on Cloud Run, Firestore

### Achievements

1. Coded the three Rule-36 checks (pitching in line, impact in line, hitting stumps)
2. Built the Three.js 3D-trajectory viewer inside a Flutter WebView
3. Connected the in-app capture/calibration/trim screens to the FastAPI backend with Firestore persistence

### Problems (if any):

1. None

### Task for next meeting:

1. Run the pipeline end-to-end on a real phone clip (`test3.mp4`) and on an 8-scenario synthetic sweep, then write the final report

---

**Student's Name:**

Niraj Kafle

**Supervisor Signature**

## Project Log Book Entry Sheet

**Meeting No.:** 08

**Date:** 24 May 2026

**Start Time:** 10:00 AM

**Finish Time:** 11:00 AM

### Discussion Topics:

1. Real-video end-to-end test on `test3.mp4`
2. 8-scenario synthetic-validation sweep
3. Final report writing: lessons learnt and future recommendations
4. Submission and defence preparation

### Achievements

1. Processed `test3.mp4`; pipeline returned NOT OUT on a flighted off-break with a 29-point clean track
2. Built the synthetic harness to a full ICC pitch with a wide-angle phone camera; 8/8 scenarios pass (100%)
3. Wrote the final report including the lessons-learnt and future-recommendations chapters

### Problems (if any):

1. None

### Task for next meeting:

1. Submit the final report, prepare the defence slide deck, and rehearse the live demo on the project laptop

---

**Student's Name:**

Niraj Kafle

**Supervisor Signature**